SE3002 Assignment 2 — Final Testing Report

Student Name: Wahid Ahmad

Student ID: 24P-3008

Course: SE3002 Software Quality Engineering

GitHub Repository: https://github.com/p243008/SQE-Assignment-2

---

1. Test Design Summary (Part A)

1.1 Equivalence Class Partitioning (ECP)

I designed 14 ECP test cases covering valid and invalid inputs for credit hours, outstanding dues, prerequisites, availability, and already-registered status. Each test maps to a specific Functional Requirement (FR-01 to FR-15) and is tagged with a unique ID (e.g., `ECP-01`). The expected outputs were derived strictly from the SRS.

1.2 Boundary Value Analysis (BVA)

I designed 22 BVA test cases focusing on the exact boundaries specified in the SRS:

- Current credits: -1, 0, 18, 19.
- Course credits: 0, 1, 4, 5.
- Combined credit limit: 17 (valid), 18 (exactly valid), 19 (invalid).
- Dues: -1, 0, 1.
- Capacity/Enrollment: 0, capacity-1, capacity, capacity+1.

1.3 Decision Table Testing

The `registration_decision()` function follows a strict priority order: Dues > Duplicate > Prerequisites > Credit Limit > Seat Availability. I constructed a decision table with 6 base rules and 4 priority-conflict test cases (e.g., testing that Dues overrides Duplicate registration). I also added 3 scenario tests (DT-11, DT-12, DT-13) for exact boundary executions.

1.4 Optimization and Traceability

To optimize the test suite, I consolidated exact duplicates across ECP, BVA, and Decision Tables. For example, `ECP-07`, `BVA-11`, and `DT-01` all tested a Dues amount of 1, resulting in `DUES_BLOCK`. I merged these into a single `pytest.mark.parametrize` execution tagged with all three IDs: `@pytest.mark.parametrize(..., ids=["ECP-07__BVA-11__DT-01_dues_blocks"])`. This preserved technique-to-test traceability while reducing redundant execution.

2. White-Box Testing Analysis (Part B)

2.1 Control Flow Graph and Cyclomatic Complexity

The CFG for `registration_decision()` was drawn by mapping every decision, processing, and return node. (See `docs/cfg.png`).

Convention: For the compound condition `not 0 <= course.enrolled <= course.capacity or course.capacity < 1`, I used two explicit conventions:

1. **Statement-level convention:** The entire `if` statement is treated as a single decision node.

- Nodes (N) = 17, Edges (E) = 23
- **V(G) = E - N + 2 = 8**
- Decision Nodes + 1 = 7 + 1 = 8. (Both methods agree).

2. **Predicate-node convention:** The compound condition is broken down into its atomic conditions (A, B, C), accounting for Python's short-circuiting.

- Nodes (N) = 19, Edges (E) = 27
- **V(G) = E - N + 2 = 10**
- Decision Nodes + 1 = 9 + 1 = 10. (Both methods agree).

2.2 Independent Basis Paths

I identified 10 independent basis paths representing the minimum set of paths needed to execute every edge in the predicate-node CFG. These paths map to representative tests (WB-P1 to WB-P10), targeting branch outcomes such as the DUES_BLOCK return and the short-circuit failures of the capacity validation.

2.3 MC/DC (Modified Condition/Decision Coverage)

For the compound validation `not 0 <= enrolled <= capacity or capacity < 1`:

- **Atomic A:** `0 <= enrolled`
- **Atomic B:** `enrolled <= capacity`
- **Atomic C:** `capacity < 1`

Using short-circuit logic (B is Not-Evaluated if A is False; C is Not-Evaluated if A or B is False), I demonstrated N+1 (4) test cases (WB-MC1 to WB-MC4). By showing the independence pairs (e.g., WB-MC1 vs WB-MC4 differs only in C, flipping the outcome), I manually proved the independent effect of each atomic condition without relying on `pytest-cov`.

3. Coverage Analysis

3.1 Initial Coverage (Black-Box Only)

- **Statements:** 92%
- **Branches:** 92%
- **Missing lines:** 51, 59, 71.
- Analysis: Line 51 was unreachable because no black-box test provided `dues > 1000` (which exposed Defect D3). Lines 59 and 71 were unreachable because the waitlist branch was dead code due to Defect D4 (`enrolled > capacity` instead of >=).

3.2 Final Coverage (Black-Box + White-Box + Defect Fixes)

After adding `tests/test_whitebox.py` and applying the defect fixes:

- **Statements:** 100%
- **Branches:** 100%
- Analysis: Line 51 was covered by the added White-Box test `WB-P5` (`dues=1001`). Lines 59 and 71 were covered by the original Black-Box tests (`SC-02`, `SC-03`) once Defect D4 was fixed, restoring the `WAITLISTED` execution path.

4. Defects, Issues, and Fix Status

A total of 5 distinct defects were found by the black-box test suite. All 5 were documented as GitHub Issues and fixed in a separate feature branch (`defect-fixes`).

Defect ID	Requirement	Description	Failed Test IDs	Status
D1	FR-01	`check_credit_limit` rejected exactly 18 credits because it used < instead of <=.	BVA-03, BVA-18, DT-11, SC-10	Fixed

Defect ID	Requirement	Description	Failed Test IDs	Status
D2	FR-03	`prerequisites_satisfied` returned True if any prereq was met, instead of all.	ECP-06, DT-12	Fixed
D3	FR-04	`DUES_BLOCK` only triggered if `dues > 1000` instead of `dues > 0`.	ECP-07, DT-07, SC-04	Fixed
D4	FR-08	Waitlist was unreachable because it checked `enrolled > capacity` instead of >=.	ECP-10, BVA-21, SC-02, SC-03	Fixed
D5	FR-13	`withdraw_course` failed to decrement the `course.enrolled` counter.	SC-08	Fixed

Issue Tracking: Issues #3, #4, #5, #6, #7.

Test Results Before Fixes: 14 failed, 58 passed, 2 skipped.

Test Results After Fixes: 72 passed, 2 skipped.

5. GitHub Collaboration & Links

- **Repository:** https://github.com/p243008/SQE-Assignment-2
- **PR #1 (Black-box tests):** https://github.com/p243008/SQE-Assignment-2/pull/1
- Evidence: Preserved failing Actions run (14 failed) demonstrating defects.
- **PR #2 (White-box tests):** https://github.com/p243008/SQE-Assignment-2/pull/2
- Evidence: Increased coverage, documented dead code.
- **PR #3 (Defect Fixes):** https://github.com/p243008/SQE-Assignment-2/pull/3
- Evidence: Passed Actions run (72 passed, 100% coverage), linked to all 5 issues.

Team Contributions:

Since I completed this assignment individually, I was responsible for all parts of the project. I designed the black-box test cases (ECP, BVA, Decision Tables) and white-box analyses (CFG, MC/DC, Basis Paths). I wrote both `test_blackbox.py` and `test_whitebox.py`, found and fixed the 5 defects in `registration.py`, and set up the GitHub Issues, Pull Requests, and Actions workflow.